

```
MariaDB [employee]> SELECT * FROM departments;
```

id	department_name	manager_id
1	Assembly	1
2	Quality Control	2
3	Machining	3
4	Logistics	4
5	Maintenance	5
6	R&D	6
7	Procurement	7
8	Health & Safety	8
9	Packaging	9
10	Inventory	10
11	Welding	11
12	Plant Management	12

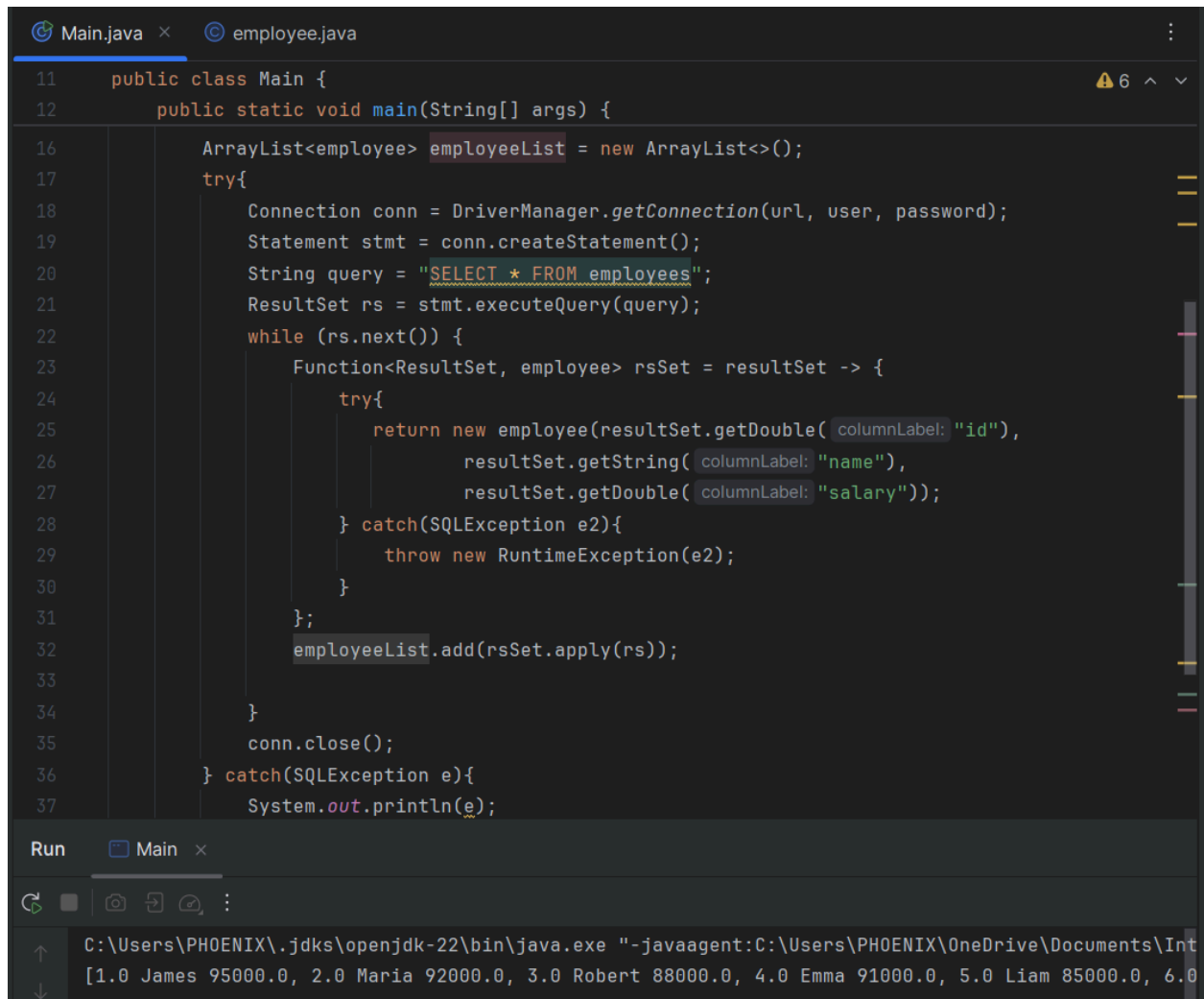
```
12 rows in set (0.043 sec)
```

```
MariaDB [employee]> SELECT * FROM employees;
```

id	name	salary	department_id	manager_id
1	James	95000	1	NULL
2	Maria	92000	2	NULL
3	Robert	88000	3	NULL
4	Emma	91000	4	NULL
5	Liam	85000	5	NULL
6	Olivia	110000	6	NULL
7	Noah	78000	7	NULL
8	Ava	82000	8	NULL
9	William	75000	9	NULL
10	Sophia	84000	10	NULL
11	Ethan	89000	11	NULL
12	Isabella	125000	12	NULL
13	Charlotte	53936	3	4
14	Ella	50213	8	9
15	Scarlett	44472	8	6
16	Ava	56608	7	11
17	Scarlett	64518	6	10
18	Charlotte	44713	10	3
19	Evelyn	65731	3	8
20	Mia	50012	NULL	7

After doing this, you should populate the

following list: List<Employee> employeesList=new ArrayList<Employee>();



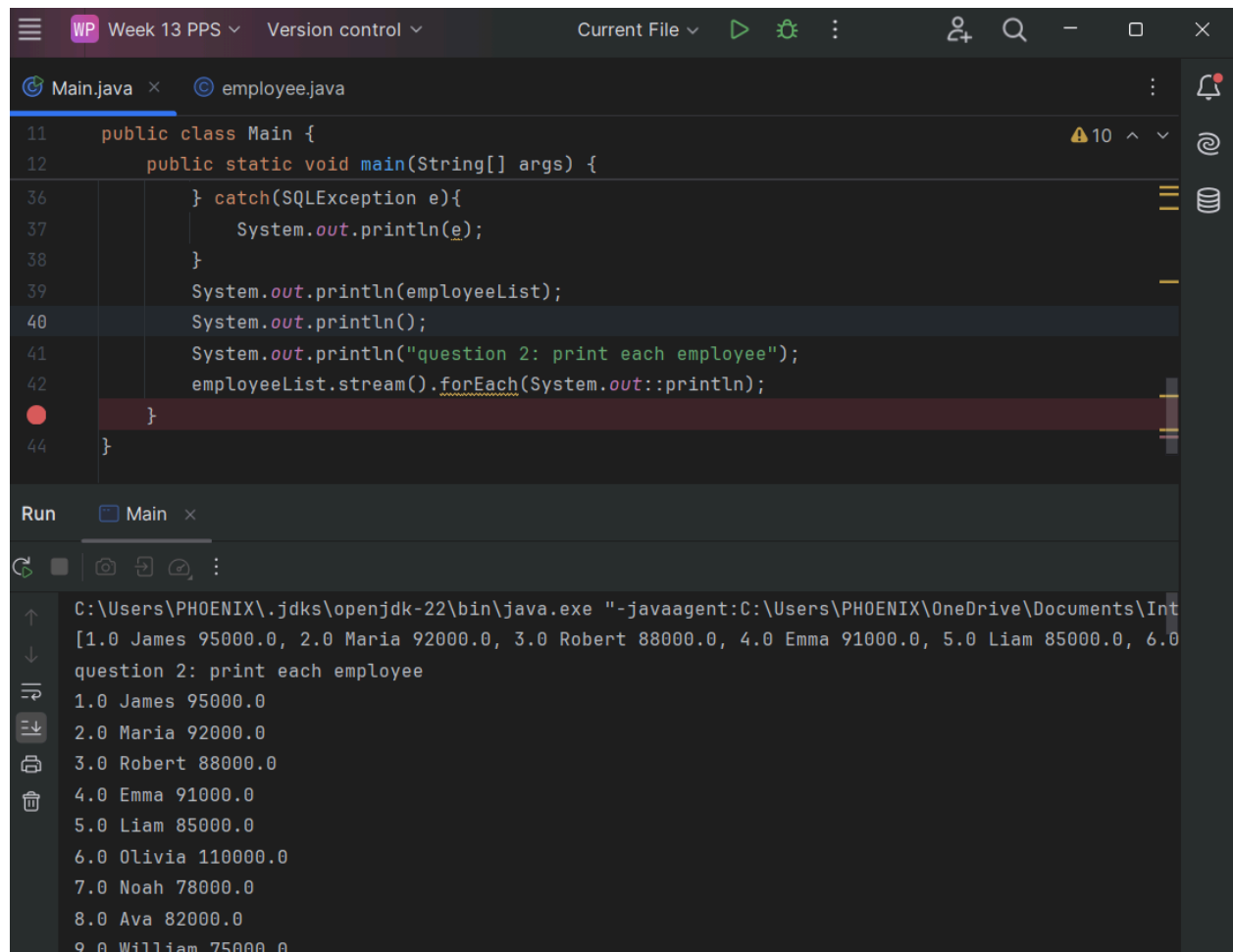
The screenshot shows an IDE with two tabs: 'Main.java' and 'employee.java'. The 'Main.java' tab is active, displaying the following code:

```
11 public class Main {
12     public static void main(String[] args) {
16         ArrayList<employee> employeeList = new ArrayList<>();
17         try{
18             Connection conn = DriverManager.getConnection(url, user, password);
19             Statement stmt = conn.createStatement();
20             String query = "SELECT * FROM employees";
21             ResultSet rs = stmt.executeQuery(query);
22             while (rs.next()) {
23                 Function<ResultSet, employee> rsSet = resultSet -> {
24                     try{
25                         return new employee(resultSet.getDouble( columnLabel: "id"),
26                             resultSet.getString( columnLabel: "name"),
27                             resultSet.getDouble( columnLabel: "salary"));
28                     } catch(SQLException e2){
29                         throw new RuntimeException(e2);
30                     }
31                 };
32                 employeeList.add(rsSet.apply(rs));
33             }
34             conn.close();
35         } catch(SQLException e){
36             System.out.println(e);
37         }
```

Below the code editor, there is a 'Run' button and a console window. The console window shows the output of the program:

```
C:\Users\PHOENIX\.jdk\openjdk-22\bin\java.exe "-javaagent:C:\Users\PHOENIX\OneDrive\Documents\Int
[1.0 James 95000.0, 2.0 Maria 92000.0, 3.0 Robert 88000.0, 4.0 Emma 91000.0, 5.0 Liam 85000.0, 6.0
```

2. Print a list of all Employees



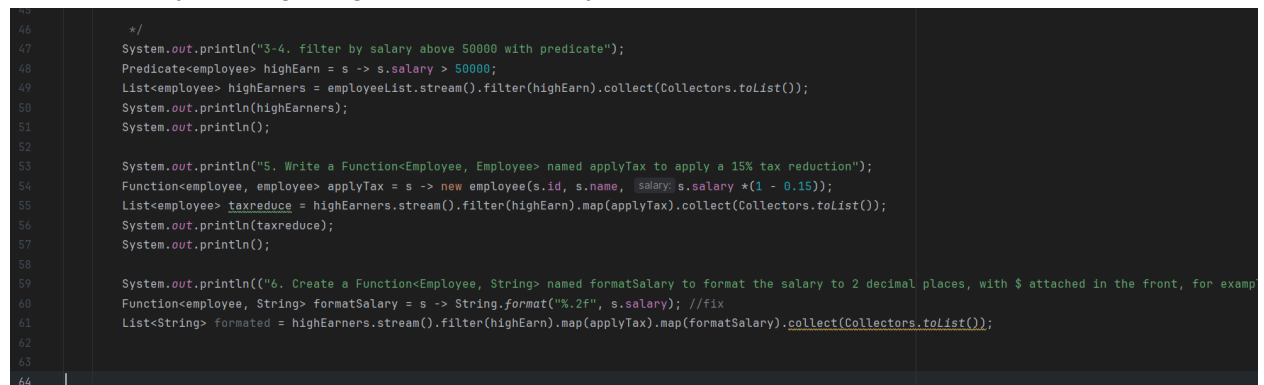
The screenshot shows an IDE with two tabs: 'Main.java' and 'employee.java'. The 'Main.java' tab is active, displaying the following code:

```
11 public class Main {
12     public static void main(String[] args) {
36         } catch(SQLException e){
37             System.out.println(e);
38         }
39         System.out.println(employeeList);
40         System.out.println();
41         System.out.println("question 2: print each employee");
42         employeeList.stream().forEach(System.out::println);
43     }
44 }
```

Below the code editor, the 'Run' tab is active, showing the output of the program:

```
C:\Users\PHOENIX\.jdk\openjdk-22\bin\java.exe "-javaagent:C:\Users\PHOENIX\OneDrive\Documents\Int
[1.0 James 95000.0, 2.0 Maria 92000.0, 3.0 Robert 88000.0, 4.0 Emma 91000.0, 5.0 Liam 85000.0, 6.0
question 2: print each employee
1.0 James 95000.0
2.0 Maria 92000.0
3.0 Robert 88000.0
4.0 Emma 91000.0
5.0 Liam 85000.0
6.0 Olivia 110000.0
7.0 Noah 78000.0
8.0 Ava 82000.0
9.0 William 75000.0
```

3. Define a Predicate to filter employees with salary > \$50000. Apply this Predicate to filter the List of Employees to get highEarnings<Employee> who earn more than \$50000.



The screenshot shows an IDE with a single tab containing the following code:

```
46     */
47     System.out.println("3-4. filter by salary above 50000 with predicate");
48     Predicate<employee> highEarn = s -> s.salary > 50000;
49     List<employee> highEarnings = employeeList.stream().filter(highEarn).collect(Collectors.toList());
50     System.out.println(highEarnings);
51     System.out.println();
52
53     System.out.println("5. Write a Function<Employee, Employee> named applyTax to apply a 15% tax reduction");
54     Function<employee, employee> applyTax = s -> new employee(s.id, s.name, salary: s.salary *(1 - 0.15));
55     List<employee> taxreduce = highEarnings.stream().filter(highEarn).map(applyTax).collect(Collectors.toList());
56     System.out.println(taxreduce);
57     System.out.println();
58
59     System.out.println("6. Create a Function<Employee, String> named formatSalary to format the salary to 2 decimal places, with $ attached in the front, for examp
60     Function<employee, String> formatSalary = s -> String.format("%.2f", s.salary); //fix
61     List<String> formatted = highEarnings.stream().filter(highEarn).map(applyTax).map(formatSalary).collect(Collectors.toList());
62
63
64
```

```
3-4. filter by salary above 50000 with predicate
[1.0 James 95000.0, 2.0 Maria 92000.0, 3.0 Robert 88000.0, 4.0 Emma 91000.0, 5.0 Liam 85000.0, 6.0 Olivia 110000.0, 7.0 Noah 78000.0, 8.0 Ava 82000.0, 9.0 William 75000.0, 10.0 Charles 68000.0]

5. Write a Function<Employee, Employee> named applyTax to apply a 15% tax reduction
[1.0 James 80750.0, 2.0 Maria 78200.0, 3.0 Robert 74800.0, 4.0 Emma 77350.0, 5.0 Liam 72250.0, 6.0 Olivia 93500.0, 7.0 Noah 66300.0, 8.0 Ava 69700.0, 9.0 William 63750.0, 10.0 Charles 58300.0]

6. Create a Function<Employee, String> named formatSalary to format the salary to 2 decimal places, with $ attached in the front, for example: $52000.30
[80750.00, 78200.00, 74800.00, 77350.00, 72250.00, 93500.00, 66300.00, 69700.00, 63750.00, 71400.00, 75650.00, 106250.00, 45845.60, 42681.05, 48116.80, 54840.30, 55871.35, 50160.00]

Process finished with exit code 0
```